BDI Developer Guide

Getting Started

Prerequisites

Required Tools

- **GCC 13+** or **Clang 16+** (C23 support required)
- **Make** (build system)
- **Git** (version control)
- **Python 3.8+** (for build scripts)

Optional Tools

- **clang-format** (code formatting)
- **clang-tidy** (static analysis)
- **cppcheck** (additional static analysis)
- **valgrind** (memory debugging)
- **gdb** or **lldb** (debugging)
- **perf** (performance profiling)
- **lcov** (code coverage)

Installing Dependencies

Ubuntu/Debian:

```
sudo apt-get update
sudo apt-get install -y \
    gcc-13 g++-13 \
    clang-16 clang-tidy-16 clang-format-16 \
    make cmake \
    git \
    python3 python3-pip \
    cppcheck valgrind gdb \
    linux-tools-generic \
    lcov
```

Fedora/RHEL:

```
sudo dnf install -y \
    gcc gcc-c++ \
    clang clang-tools-extra \
    make cmake \
    git \
    python3 python3-pip \
    cppcheck valgrind gdb \
    perf \
    lcov
```

macOS:

```
brew install gcc@13 llvm make git python3 cppcheck valgrind lcov
```

Repository Setup

Clone Repository

```
git clone https://github.com/Mecca-Research/The-Binary-Decomposition-Interface.git
cd The-Binary-Decomposition-Interface
```

Sparse Checkout (Recommended)

```
git sparse-checkout init --cone
git sparse-checkout set C tests docs .github
```

Configure Git

```
git config user.name "Your Name"
git config user.email "your.email@example.com"
```

Initial Build

Build Debug Version

```
make BUILD_MODE=debug
```

Run Tests

```
cd tests
make
make run
```

Verify Installation

```
# Check compiler version
gcc --version | grep "gcc (GCC) 13"

# Verify C23 support
echo "int main() { void* p = nullptr; return 0; }" | gcc -std=c2x -x c - -o /tmp/test
&& echo "C23 supported"
```

Development Workflow

Branch Strategy

Main Branches

- `main` : Stable production code
- `develop` : Integration branch for features

Feature Branches

```
# Create feature branch
git checkout -b feature/my-feature main

# Work on feature
# ... make changes ...

# Commit changes
git add .
git commit -m "feat: add new feature"

# Push to remote
git push origin feature/my-feature

# Create pull request
```

Branch Naming Convention

- `feature/` : New features
- `bugfix/` : Bug fixes
- `refactor/` : Code refactoring
- `docs/` : Documentation updates
- `test/` : Test additions/improvements

Commit Message Convention

Follow Conventional Commits (https://www.conventionalcommits.org/):

```
<type>(<scope>): <subject>

<body>

<footer>
```

Types:
- `feat` : New feature
- `fix` : Bug fix
- `docs` : Documentation changes
- `style` : Code style changes (formatting)
- `refactor` : Code refactoring
- `test` : Test additions/changes
- `chore` : Build process or auxiliary tool changes
- `perf` : Performance improvements

Examples:

```
git commit -m "feat(compiler): add nullptr support to lexer"
git commit -m "fix(vm): resolve stack overflow in recursive calls"
git commit -m "docs(api): update API contracts for device module"
git commit -m "refactor(kernel): modernize scheduler with C23 features"
```

Code Style

Formatting

The project uses **clang-format** with configuration in `.clang-format`.

Format Single File:

```
clang-format -i C/kernel/device/device.c
```

Format All Files:

```
make format
```

Check Formatting:

```
make format-check
```

Naming Conventions

Functions: `snake_case`

```
int device_initialize(void);
void* memory_allocate(size_t size);
```

Variables: `snake_case`

```
int error_code = 0;
Device* current_device = nullptr;
```

Constants: `UPPER_SNAKE_CASE`

```
#define MAX_DEVICES 256
#define DEFAULT_TIMEOUT 1000
```

Types: `PascalCase`

```
typedef struct Device Device;
typedef enum DeviceType DeviceType;
```

Macros: `UPPER_SNAKE_CASE`

```
#define ASSERT(condition) ...
#define LOG_ERROR(msg) ...
```

Code Organization

File Structure:

```c
// Header comment
// ============================================================================
// DESC: Brief description of file purpose
// ============================================================================

// System includes
#include <stdio.h>
#include <stdlib.h>

// Project includes
#include "device.h"
#include "scheduler.h"

// Constants
#define MAX_BUFFER_SIZE 4096

// Type definitions
typedef struct Buffer {
    char* data;
    size_t size;
} Buffer;

// Static (private) function declarations
static int internal_helper(void);

// Public function implementations
int public_function(void) {
    // Implementation
}

// Static function implementations
static int internal_helper(void) {
    // Implementation
}
```

Header Guards:

```c
#ifndef BDI_MODULE_NAME_H
#define BDI_MODULE_NAME_H

// Header content

#endif // BDI_MODULE_NAME_H
```

C23 Features Usage

nullptr

```c
// ✅ Good
void* ptr = nullptr;
if (ptr == nullptr) { ... }

// ❌ Bad
void* ptr = NULL;
if (ptr == NULL) { ... }
```

Attributes

```c
// nodiscard - function result should not be ignored
[[nodiscard]] int allocate_resource(void);

// maybe_unused - suppress unused warnings
[[maybe_unused]] static int debug_helper(void);

// noreturn - function never returns
[[noreturn]] void fatal_error(const char* msg);

// fallthrough - intentional switch fallthrough
switch (value) {
    case 1:
        do_something();
        [[fallthrough]];
    case 2:
        do_something_else();
        break;
}
```

static_assert

```c
// Compile-time checks
static_assert(sizeof(int) == 4, "int must be 4 bytes");
static_assert(sizeof(Device) <= 256, "Device struct too large");
static_assert(_Alignof(Buffer) >= 8, "Buffer must be 8-byte aligned");
```

typeof and auto

```c
// Type inference
int x = 42;
typeof(x) y = x;  // y is int

auto z = 3.14f;   // z is float

// Generic macros
#define MAX(a, b) ({ \
    typeof(a) _a = (a); \
    typeof(b) _b = (b); \
    _a > _b ? _a : _b; \
})
```

Testing

Writing Tests

Test File Structure:

```c
#include "test_framework.h"
#include "module_to_test.h"

// Test function
static bool test_feature_name(void) {
    // Setup
    Device* device = device_create();

    // Test
    int result = device_initialize(device);

    // Assertions
    ASSERT_EQ(result, 0, "initialization should succeed");
    ASSERT_NOT_NULL(device, "device should not be nullptr");

    // Cleanup
    device_destroy(device);

    return true;
}

int main(void) {
    TEST_INIT();

    run_test("test_feature_name", test_feature_name);

    TEST_SUMMARY();
}
```

Running Tests

All Tests:

```
cd tests
make run
```

Single Test:

```
cd tests
./test_nullptr
```

With Sanitizers:

```
cd tests
make clean
make CFLAGS="-std=c2x -g -fsanitize=address"
make run
```

Test Coverage

Generate Coverage Report:

```
cd tests
make clean
make CFLAGS="-std=c2x -g --coverage" LDFLAGS="--coverage"
make run
lcov --capture --directory . --output-file coverage.info
lcov --remove coverage.info '/usr/*' --output-file coverage.info
genhtml coverage.info --output-directory coverage_html
```

View Report:

```
firefox coverage_html/index.html
```

Debugging

GDB Basics

Start Debugging:

```
gdb ./build/bin/program
```

Common Commands:

```
(gdb) break main              # Set breakpoint at main
(gdb) break device.c:42       # Set breakpoint at line 42
(gdb) run                     # Start program
(gdb) next                    # Step over
(gdb) step                    # Step into
(gdb) continue                # Continue execution
(gdb) print variable          # Print variable value
(gdb) backtrace               # Show call stack
(gdb) info locals             # Show local variables
(gdb) quit                    # Exit GDB
```

Valgrind

Memory Leak Detection:

```
valgrind --leak-check=full --show-leak-kinds=all ./program
```

Memory Error Detection:

```
valgrind --tool=memcheck --track-origins=yes ./program
```

Address Sanitizer

Build with ASan:

```
make BUILD_MODE=debug SANITIZER=address
```

Run:

```
./build/bin/program
```

Profiling

perf

Record Profile:

```
perf record -g ./program
```

View Report:

```
perf report
```

Annotate Source:

```
perf annotate
```

gprof

Build with Profiling:

```
make CFLAGS="-pg" LDFLAGS="-pg"
```

Run and Generate Report:

```
./program
gprof ./program gmon.out > analysis.txt
```

Static Analysis

clang-tidy

Run on Single File:

```
clang-tidy C/kernel/device/device.c -- -std=c2x -I.
```

Run on All Files:

```
make clang-tidy
```

Fix Issues Automatically:

```
clang-tidy -fix C/kernel/device/device.c -- -std=c2x -I.
```

cppcheck

Run Analysis:

```
cppcheck --enable=all --suppress=missingIncludeSystem C/
```

Generate Report:

```
cppcheck --enable=all --xml --xml-version=2 C/ 2> cppcheck-report.xml
```

Documentation

Code Documentation

Function Documentation:

```
/**
 * @brief Initialize the device subsystem
 *
 * This function must be called before any device operations.
 * It enumerates available devices and initializes their backends.
 *
 * @return 0 on success, negative error code on failure
 * @retval 0 Success
 * @retval -1 Initialization failed
 * @retval -2 No devices found
 *
 * @note This function is not thread-safe
 * @warning Must be called from main thread only
 *
 * @see device_cleanup()
 */
int device_init(void);
```

Struct Documentation:

```
/**
 * @brief Device structure
 *
 * Represents a computational device (CPU, GPU, FPGA, etc.)
 */
typedef struct Device {
    int id;                     ///< Unique device identifier
    DeviceType type;            ///< Device type
    DeviceCapabilities caps;    ///< Device capabilities
    void* backend_data;         ///< Backend-specific data
} Device;
```

Markdown Documentation

Update Documentation:

```
# Edit documentation
vim docs/api_contracts.md

# Preview (if using grip)
grip docs/api_contracts.md
```

Performance Optimization

Profile-Guided Optimization

Step 1: Generate Profile:

```
make BUILD_MODE=pgo-gen
./build/bin/program < benchmark_input.txt
```

Step 2: Build with Profile:

```
make BUILD_MODE=pgo-use
```

Optimization Tips

1. **Hot Path Optimization**:
   - Identify hot paths with profiling
   - Inline critical functions
   - Reduce branching
   - Improve cache locality

2. **Memory Optimization**:
   - Use memory pools for frequent allocations
   - Align data structures to cache lines
   - Minimize pointer chasing
   - Use structure packing wisely

3. **Compiler Hints**:

```
// Likely/unlikely branches
if (__builtin_expect(error != 0, 0)) {
    handle_error();
}

// Prefetch data
__builtin_prefetch(data, 0, 3);

// Assume alignment
void* aligned_ptr = __builtin_assume_aligned(ptr, 64);
```

Continuous Integration

GitHub Actions

The project uses GitHub Actions for CI/CD (`.github/workflows/ci.yml`).

Workflow Triggers:
- Push to `main` or `refactor/**` branches
- Pull requests to `main`

CI Jobs:
1. **Build and Test**: Multiple compilers and build modes
2. **Sanitizer Checks**: ASan, UBSan, LSan
3. **Format Check**: Code formatting validation
4. **Static Analysis**: clang-tidy and cppcheck

Local CI Simulation:

```
# Run all checks locally
make clean
make BUILD_MODE=debug
cd tests && make run
make clang-tidy
make cppcheck
make format-check
```

Common Tasks

Adding a New Module

1. **Create Directory Structure**:

```
mkdir -p C/new_module
touch C/new_module/new_module.h
touch C/new_module/new_module.c
```

1. **Write Header**:

```
// C/new_module/new_module.h
#ifndef BDI_NEW_MODULE_H
#define BDI_NEW_MODULE_H

int new_module_init(void);
void new_module_cleanup(void);

#endif // BDI_NEW_MODULE_H
```

1. **Write Implementation**:
   ```c
   // C/new_module/new_module.c

# include “new_module.h”

# include
   ```